Assignment

Topics Covered: Introduction to FastAPI | Installation & Project Setup | Creating the First API | GET Method | Running the Server (Uvicorn) | API Endpoints

Question 1: What is FastAPI and why is it used to build backend APIs? Explain how it differs from a normal Python script that simply runs once and stops.

Answer:

FastAPI is a modern, fast, and easy-to-use Python web framework used to build backend APIs,. It allows different applications, such as websites and mobile apps, to communicate with the backend server and exchange data.

Why is it used to build backend APIs ?

FastAPI is used because it provides high performance, automatic data validation, support for asynchronous programming, and automatic API documentation.

Normal Python script that simply runs once and stops.

A normal Python script runs its instructions once and stops after completing the task. In contract, a FastAPI application runs continuously as a web server. It waits for HTTP requests, processes them, and sends responses whenever users access its API endpoints.

Question 2: Explain the difference between traditional web development and FastAPI-based API development. Support your answer with an example of what each approach returns to the client.

Answer:

Traditional web development and FastAPI-based API development are two different approaches used to build web applications.

Traditional Web Development :

Traditional web development focuses on creating complete web pages using HTML, CSS, javaScript, and server-side technologies. When a client sends a request to the server, the server processes the request and returns and returns an HTML page that is displayed in the browser.

Example:

```
<!DOCTYPE html>
<html>
<body>
        <h1>Welcome User!</h1>
        <p>This is my website.</p>
</body>
</html>
```

Output : The server returns an HTML page that displays “Welcome User!” in the browser.

FastAPI-Based API Development :

FastAPI-based API development focuses on creating APIs that allow different applications to communicate with each other. Instead of returning complete web pages, FastAPI generally returns structured data in JSON format. This data can be used by websites, mobile applications, and other software.

Example :

```
From fastapi import FastAPI

app = FastAPI()

@app.get(“/”)
Def home():
        return {“message” : “Welcome User !”}
```

Output :

```
{
        “message” : “Welcome User !”
}
```

Question 3: List the steps required to set up a FastAPI project locally, starting from installing Python to running the server with Uvicorn. Explain the purpose of a virtual environment in this setup.

Answer:

To set up a FastAPI project locally, we need to install Python, create a project folder, set up a virtual environment, install FastAPI and Unicorn, create a Python file, and run the server.

Steps to set Up a FastAPI Project :

Step 1 : Install Python

First, download and install Python from official website :

https://www.python.org/

Check whether Python is installed successfully by running :

```
python - -version
```

Step 2 : Create a Project Folder

Create a folder for the FastAPI project and open it in the terminal.

```
mkdir fastapi_project
cd fastapi_project
```

Step 3. Create a virtual Environment

Create a virtual environment using the following command :

```
python -m venv venv
```

Step 4 : Active the virtual Environment

For windows :

```
venv \ script \ activate
```

For Linux or macOS :

```
Source veny / bin / activate
```

Step 5. Install FastAPI and Unicorn

Install the required packages using pip :

```
pip install fastapi unicorn
```

- FastAPI : Used to create backend APIs.
- Unicorn : Used as an ASGI server to run the FastAPI application.

Step 6 : Create a Python File

Create a file named main.py and write the following code :

```
From fastapi import FastAPI

app = FastAPI ()

@app.get (“/”)
def home() :
        return {“message” : “Hello, FastAPI ! “}
```

Step 7 : Run the FastAPI server

Run the application using Unicorn :

```
        Unicorn main : app - -reload
```

Here :

- Main refers to the Python file name.
- app refers to the FastAPI application object.
- - -reload automatically restarts the server when code changes.

Step 8 : Open the application

Open the following URL in your browser :

http://127.0.0.1:8000

To access automatic API documentation, open:

http://127.0.0.1:8000/docs

Purpose of a virtual Environment :

A virtual environment is an isolated environment that allows us to install and manage project-specific Python packages separately.

Question 4: What is Uvicorn, and why does a FastAPI application need an ASGI server to run? Explain the role of Uvicorn in the request-response flow.

Answer:

Uvicorn is a fast and highweight ASGI (Asynchronous Server Gateway Interface) server used to run FastAPI applications. It acts as a bridge between the client and the FastAPI application.

A FastAPI application needs an ASGI server because FastAPI supports asynchronous programming and requires a server to receive and handle HTTP requests. Uvicorn provides this server functionality and helps FastAPI communicate with client efficiently.

Role of Unicorn in the Request-Response Flow :

1. **Client Request :** A client, such as web browser or mobile application, sends an HTTP request to the server.
2. **Uvicorn Receives Request :** Uvicorn receives the incoming request and forwards it to the FastAPI application.
3. **Request Processing :** FastAPI processes the request and executes the required function or API endpoint.
4. **Response Generation :** FastAPI generates a response, usually in JSON format.
5. **Uvicorn Sends Response :** Uvicorn sends the response back to the client.

Question 5: Write the minimal FastAPI code required to create an application with one route at / that returns the message "Hello, FastAPI!". Explain each line of the code.

Answer:

The following is the minimal FastAPI code to create an application with one route (/) that returns the message “ Hello, FastAPI ! “.

Code :

```
from fastapi import FastAPI

app = FastAPI ()

@app.get(“/”)
def home() :
        return {“message” : “Hello, FastAPI !”}
```

Explain of Each Line :

1. **From fastapi import FastAPI :-** Import the FastAPI class from the FastAPI library.
2. **app = FastAPI() :-** Creates an instance of the FastAPI application.
3. **@app.get(“/”) :-** Defines a GET route at the root URL (/). It handles requests sent to this endpoint.

4. **def home() :-** Defines a function named home that executes when the root route is accessed.
5. **return {“message”: “Hello, FastAPI !”} :-** Returns a JSON response containing the message “Hello, FastAPI !”.

Output :

```
{
	“message” : “Hello, FastAPI !”
}
```

Question 6: Explain the GET method — what it is used for, and why it is considered a safe, read-only operation. Give two real-world examples of data that could be returned by a GET endpoint in a student management system.

Answer:

The GET method is an HTTP method used to retrieve or request data from a server. In FastAPI, it is used to create API endpoints that allow clients to access information stored in an application or database.
The GET method is considered a safe, read-only operation because its main purpose is only to retrieve data. It does not modify, delete, or update the information stored on the server.

Why is GET considered a safe, read-only operation ?

1. It is used only to retrieve existing information.
2. It does not change the data stored in the database.
3. It does not create or delete records.
4. It allows users to access information without affecting the original data.

Two Real-world Examples of GET method in a Student Management System :

1. **Retrieve Student Details :** A GET endpoint can return a student’s personal information, such as name, roll number, age, course, and email address.

Code:

```
@app.get(“/students/{student_id}”)
def get_student_details(student_id : int):
	return {
		“student_id” : student_id,
		“name” : “Rahul”,
		“roll_no” : 101,
		“course” : “BCA”
	}
```

Output :

URL : http://127.0.0.1:8000/students/101

```
{
        “student_id” : 101,
        “name” : “Rahul”,
        “roll_no” : 101,
        “course” : “BCA”
}
```

2. **Retrieve Student Attendance :** A GET endpoint can return a student’s attendance records, including total classes, classes attended, and attendance percentage.

```
@app.get(“/students/{student_id}/attendance”)
def get_student_attendance(student_id: int):
        return {
                “student_id” : student_id,
                “total_classes” :  100,
                “attended_classes” : 85,
                “attendance_percentage” : 85
        }
```

Output :

```
{
        “student_id” : 101,
        “total_classes” : 100,
        “attended_classes” : 85,
        “attendance_percentage” : 85
}
```

Question 7: Write FastAPI code to create the following GET endpoints in one main.py file:
1. / – returns a welcome message
2. /about – returns the app name and a short description
3. /courses – returns a list containing the names of three courses

Answer:

The following FastAPI code creates three GET endpoints in a single main.py file.

Code :

```
from fastapi import FastAPI

app = FastAPI ()

# 1. Welcome Message

app.get("/")
def home():
    return {"message" : "Welcome to FastAPI"}

# 2. About Application

@app.get("/about")
def about():
    return {
        "app_name" : "Student Management System",
        "description" : "An application to manage student records and courses."
    }

# 3. List of courses

@app.get("/courses")
def courses():
    return {
        "courses" : ["BCA", "MCA", "BBA"]
    }
```

Explanation :

1. **@app.get("/") :-** Returns a welcome message when the user visits the root URL.
2. **@app.get("/about") :-** Returns the application name and its about description.
3. **@app.get("/courses") :-** Returns a list containing the names of three courses.

Output:

1. Endpoint: /

```
{
    "message" : "Welcome to FastAPI"
```

}

2. Endpoint : /about

{
“app_name” : “student Management System”,
“description” : “An application to manage student records and courses.”
}

3. Endpoint : /courses

{
“courses”: [“BCA”, “MCA”, “BBA”]
}

To Run the Application:

Unicorn main : app --reload

Open http: //127.0.0.1: 8000/docs in your browser to test all three GET endpoints.

Question 8: Explain the command used to run a FastAPI server locally using Uvicorn. Describe the meaning of main:app and the purpose of the --reload option.

Answer:

The command used to run a FastAPI application locally using Uvicorn is:

uvicorn main : app - -reload

This command starts the FastAPI application on a local development server, usually at http://127.0.0.1:8000.

Explanation of the Command:

1. **Uvicorn :-** It is an ASGI server used to run the FastAPI application.
2. **Main :-** It represents the name of the python file (main.py) where the FastAPI application is defined.
3. **app :-** It represents the FastAPI application object created using app = FastAPI() inside the main.py file.
4. **- -reload:-** This option automatically restarts the server whenever changes are made to the Python code. It is mainly used during development.

Question 9: What is an API endpoint? Using the endpoints /, /about, /student and /students as examples, explain what makes an endpoint name clear and meaningful, and what happens when a requested endpoint does not exist in main.py.

Answer:

An API endpoint is a specific URL or path through which a client communicates with a backend application to request or exchange data. In FastAPI, endpoints are created using HTTP methods such as GET, POST, PUT, and DELETE.

Examples of API Endpoints:

1. / - Used to display a welcome message.
2. / about - Used to provide information about the application.
3. / student - Used to retrieve information about a single student.
4. / students - Used to retrieve information about multiple students.

What Makes an Endpoint Name Clear and Meaningful ?

- An endpoint name should clearly describe the data or resource it represents.
- It should be short, simple, and easy to understand.
- It should be short, simple, and easy to understand.
- Generally, singular names represent one resource (/ student). While plural names represent multiple resources (/ students).

What Happens When a Requested Endpoint Does Not Exist in main.py ?

If a client requests an endpoint that has not been defined in main.py, FastAPI returns an HTTP 404 Not Found error. It means that the requested URL does not exist in the application.

Example:

If /teachers is not defined in main.py and client requests:

http://127.0.0.1:8000/teachers

FastAPI returns:

```
{
        “detail” : “Not Found”
}
```

Question 10: Build a small Library API using FastAPI. Write the complete main.py code that creates the following GET endpoints:
1. / – returns a welcome message for the Library API
2. /library-info – returns the library name, total number of books, and working hours
3. /books – returns a list of at least 3 books, where each book has an id, title and author
4. /available-books – returns only the books that are currently available (choose any 2 from the list above)
5. /featured-book – returns one recommended book with its id, title and reason for the recommendation
Also write the Uvicorn command used to run this application in Google Colab, and the Python requests code used to test the /books endpoint.

Answer:

The following FastAPI code creates a small Library API with five GET endpoints. Each endpoint returns specific information about the library and its books in JSON format.

1. Complete FastAPI code (main.py)

```
from fastapi import FastAPI

app = FastAPI()

# Book Data

books = [
        {
                “id” :  1,
                “title” :  “Python Programming”,
                “author” :  “John Smith”,
                “available” :  True
        },
        {
                “id” :  2,
                “title” :  “Data Science Handbook”,
                “author” :  “Jake VanderPlas”,
                “avalable” : True
        },
        {
                “id” :  3,
                “title” :  “Machine Learning Basics”,
                “author” :  :Andrew Ng”,
        }
]
```

```
# 1. Welcome Endpoint

@app.get("/")
def home():
    return {"message" : "Welcome to the library API"}

# 2. Library Information

@app.get("/library-info")
def library_info():
    return {
        "library_name" :  "central Public Library",
        "total_books":  len(books),
        "working_hours" :  "9 AM to 6 PM"
    }

# 3. Get All Books

@app.get("/book")
def get_books():
    returns books

# 4. Get Available Books

@app.get("/available-books")
def available_books():
    return [book for book in books if book["available"]]

# 5. Get Featured Book

@app.get("/featured-book")
def featured_book():
    return {
        "id" :  2,
        "title" :  "Data Science Handbook",
        "reason" :  "This book provides useful knowledge of data analysis and data science."
```

2. Explanation of Endpoints

1. / - Returns a welcome message for the library API.
2. / library-info - Returns the library name, total number of books, and working hours.
3. / books - Returns a list of all three books with their IDs, titles, and authors.

4. / available-books - Returns only two books that are currently available.
5. / featured-book - Returns one recommended book with its ID, title, and recommendation reason.

3. Uvicorn Command to Run in Google Colab

First, install the required libraries:

!pip install fastapi unicorn nest-asyncio pyngrok

Run the FastAPI application using:

Code:

```
import nest_asyncio
import uvicorn
from pyngrok import ngrok

nest_asyncio.apply()

public_url = ngrok.connect(8000)
```

This starts the FastAPI server on port 8000 and provides a public URL through ngrok. An ngrok authentication token may be required.

4. Python Requests code to Test /books Endpoint

Code:

```
import requests

url = "http://127.0.0.1:8000/books"

response = requests.get(url)

print("Status code:", response.status_code)
print("Books Data:", response.json())
```

Expected Output:

Books Data:


```
[
    {
        "id": 1,
        "title": "Python Programming",
        "author": "John Smith",
        "available": true
    },
    {
        "id": 2,
        "title": "Data Science Handbook",
        "author": "Jake VanderPlas",
        "available": true
    },
    {
        "id": 3,
        "title": "Machine Learning Basics",
        "author": "Andrew Ng",
        "available": false
    }
]
```